

The Tragedy of Convenience: Cascading User-Data Leakage from SMS-Delivered URLs

Muhammad Danish
University of New Mexico
Albuquerque, New Mexico, USA
mdanish@unm.edu

Enrique Sobrados
University of New Mexico
Albuquerque, New Mexico, USA
esobrados720@unm.edu

Priya Kaushik
University of Arizona
Tucson, Arizona, USA
priyakaushik@arizona.edu

Bhupendra Acharya
University of Louisiana at Lafayette
Lafayette, Louisiana, USA
bhupendra.acharya@louisiana.edu

Muhammad Saad
Circle Internet Group
Boston, Massachusetts, USA
saad@circle.com

Abdullah Mueen
University of New Mexico
Albuquerque, New Mexico, USA
mueen@unm.edu

Sazzadur Rahaman
University of Arizona
Tucson, Arizona, USA
sazz@arizona.edu

Afsah Anwar
University of New Mexico
Albuquerque, New Mexico, USA
afsah@unm.edu

Abstract

Digital Services are increasingly sending private URLs over Short Message Service (SMS) to allow users to resume sessions with a single click. While convenient, this design shifts trust from explicit authentication to a potentially vulnerable communication channel. Basically, the vulnerability lies in the assumption that the private link can only be accessed by the intended user.

In this paper, we demonstrate that this assumption can be easily violated. In particular, we show how seemingly isolated link exposure can cascade into a wider data leak. Using public SMS gateways as an ethical lens, we analyze more than 322K unique private URLs extracted from over 33 million messages across 30K+ phone numbers. Across 701 URLs, we find that at least 177 web services effectively treat private URLs as bearer credentials, enabling unauthorized access to sensitive user information (e.g., financial details, national IDs) once the link is exposed.

Alarming, we show that 125 services are potentially enumerable, i.e., a single URL can lead to a cascading effect, resulting in the data leakage of their entire user base. Moreover, we observe that 5 services that implement authentication partially reveal account information before authentication is completed and rely on lightweight parameters (e.g., date of birth, ZIP code). Even worse, in 4 out of these 5 services, the authentication is vulnerable to brute-force attacks. Further, we uncover that 20 services grant privileged access: 14 allow modification of Personally Identifiable Information (PII), 5 grant account access, and 1 allows both. We also find 84 services that expose additional PII beyond the landing page. Our disclosures led to acknowledgments from 18 services, 7 of which have already been fixed, positively impacting at least 120 million users.

CCS Concepts

• **Security and privacy** → **Web application security**; *Authentication*; *Access control*.

Keywords

SMS security, SMS-delivered URLs, bearer URLs, authentication tokens, user enumeration, PII leakage, access control

ACM Reference Format:

Muhammad Danish, Enrique Sobrados, Priya Kaushik, Bhupendra Acharya, Muhammad Saad, Abdullah Mueen, Sazzadur Rahaman, and Afsah Anwar. 2026. The Tragedy of Convenience: Cascading User-Data Leakage from SMS-Delivered URLs. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3846541>

1 Introduction

Increasing awareness of web security threats has driven the adoption of stronger authentication mechanisms [22, 33]. Stricter password policies and Multi-Factor Authentication (MFA) have improved resistance to attacks such as credential stuffing and phishing. However, these mechanisms introduce interaction overhead through multiple authentication steps, often degrading usability and leading to user fatigue [60]. To mitigate this, web services have increasingly adopted alternative trust channels to verify user legitimacy, such as private links over Short Message Service (SMS).

Digital services using private links over SMS allow users to seamlessly resume authenticated sessions. While this approach improves usability, these services offload trust to an alternative and potentially vulnerable channel. For example, private links rely on the assumption that only the intended user has access to them, which can be undermined by multiple attack vectors, such as SIM swapping [30], cellular signaling vulnerabilities that enable SMS interception or redirection [67], malicious apps with SMS read permissions, and large-scale SMS data leaks [72]. Additionally, such URLs are often short by design. Websites may deliberately use short



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.
CCS '26, The Hague, Netherlands.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
<https://doi.org/10.1145/3830454.3846541>

URLs to improve readability and fit within character limits, thereby enhancing user experience.

However, private links with predictable patterns, such as low-entropy URLs, can be exploited to make services vulnerable to large-scale account takeover or data leaks. An attacker who identifies the URL pattern can enumerate valid links through automated requests, bypassing authentication entirely without any credentials or social engineering. It is important to note that, unlike the prior case, this case requires only that the adversary know about the weakness, without requiring large-scale SMS data leaks. Additionally, to prevent PII exposure due to accidental mishandling of SMSes, services occasionally include an authentication layer, such as a One-Time Password (OTP) or date of birth. Weaknesses in this layer of authentication can also fail under enumeration and allow unauthorized access to protected user data.

In this paper, we study private URLs to understand the threats emanating from them and the range of services affected by the threats. Therefore, we use public SMS gateways as a lens to study this critical problem, considering ethical and non-invasive conditions [12, 15, 21, 41, 56, 57]. These gateways offer a lower-risk vantage point because their messages are already publicly accessible and can be observed without interacting with recipients or manipulating carrier infrastructure. At the same time, they approximate a realistic adversarial view of SMS-delivered URLs. Overall, this study answers two critical questions: (1) *How can adversaries weaponize authentication flaws in services to infiltrate their broader user base without compromising their infrastructure?* and (2) *Are such threats isolated incidents or representative of the wider ecosystem?*

Our analysis of more than 322K URLs extracted from over 33 million messages reveals concerning insights. Using a hierarchical LLM- and expert-based PII detection and validation pipeline, we identify 701 vulnerable endpoints affecting 177 services. Notably, 125 services use tokens with entropy below 64 bits and are therefore vulnerable to user enumeration, meaning that access to a single private URL can impact users beyond the intended recipient, including those who have never used SMS gateways or whose SMS data was never leaked. For instance, if Alice requests insurance quotes from a provider and the URL sent to her is later obtained by an attacker, like via an SMS gateway, the privacy of other users (e.g., Bob and Eve) who previously interacted with the provider may also be compromised. Based on publicly available user-base information for 51 of the low-entropy services, we estimate that up to 1.05 billion user records could potentially be exposed. We further uncover 4 services that employ brute-force-able authentication mechanisms (e.g., zip code or date of birth) without rate-limiting, which are commonly used when accessing sensitive information such as medical records, raising concerns about the secure deployment of widely used technologies. Additionally, we examine cases where SMS-delivered URLs grant capabilities beyond passive viewing. We find 20 services that expose privileged access: 14 allow modification of user records, 5 grant account-level access, and 1 allows both. These cases elevate the risk from confidentiality loss to integrity and impersonation risks. We also identify 84 services that expose additional PII beyond the landing page. Alarming, we are able to completely reconstruct 206 user profiles from SMS gateway links alone, without further enumeration of other users. This exposure could be exploited by attackers for downstream attacks,

including targeted spear-phishing. Finally, we present multiple case studies to better explain our findings, disclosing only the names of services that have been fixed and have not asked for anonymity.

Although the attacks presented in this work are widely known, our contribution contextualizes a novel attack surface that enables the accurate identification of target services without relying on blind discovery. For example, with knowledge harvested by observing SMSes sent through public gateways, an attacker can carry out the exploitation using consumer-grade hardware and only basic to intermediate web security knowledge. Overall, we make the following contributions:

- (1) We analyze over 322K URLs from 33M+ messages collected across public SMS gateways, providing the first large-scale view of risks stemming from frictionless SMS-based user experiences.
- (2) Using hierarchical LLM-based PII detection and expert validation, we uncover 701 vulnerable endpoints across 177 services, including user enumeration, brute-force-able authentication, privileged data exposure, and direct account compromise.
- (3) We identify 125 services with low-entropy tokens that are potentially enumerable and confirm enumeration in 13 services under bounded testing, demonstrating how one exposed URL can extend risk to other users of the service without compromising the service infrastructure.
- (4) We find 84 services that expose additional user data compared to what is visible on the landing page, amplifying privacy risks.
- (5) We show that these attacks are easy to replicate using consumer-grade setups and limited expertise. Despite responsible disclosure attempts, many providers lacked reporting channels or dismissed our findings, highlighting critical gaps in industry readiness. Our responsible disclosures have been acknowledged by 18 services, 7 of which have been fixed. Only among the ones that were user-enumerable and have been fixed, we estimated **protecting at least 120 million users**.
- (6) We identify a concerning trend during disclosure: many providers were skeptical, often dismissing reports as potential scams. Additionally, we found most providers lack formal mechanisms or public channels to report vulnerabilities, further hindering responsible disclosure efforts.

Ethical Considerations. This study was reviewed and approved by our institution’s ethics board (IRB). We treated all collected messages, URLs, screenshots, and linked content as sensitive, even when publicly available, and ultimately deleted all raw messages, URLs, and screenshots at the end of the study. We minimized harm through conservative crawling, local-only OCR/LLM processing, bounded URL validation, no form submissions or state-changing actions, strict data minimization, and responsible disclosure to affected providers with at least a 45-day remediation window. We also explicitly weighed residual risks. Given all the preventive measures we took, the benefits of conducting and publishing this work outweigh the residual risks. We estimate that the resulting remediations protected at least 120 million users from an imminent threat of personal data leakage. We discuss ethics in detail in Appendix B.

2 Threat Model

We consider an external adversary seeking unauthorized access to user PII through SMS-delivered URLs.

Capabilities. An adversary potentially obtains one or more SMS-delivered URLs through public SMS gateways [12, 15, 21, 41, 56, 57], SIM swapping [30], cellular signaling vulnerabilities that enable SMS interception or redirection [67], malicious apps with SMS-read permissions, or large-scale SMS data leaks [72]. We consider three adversarial scenarios: **A1** uses an ordinary web client to visit the URL, follow redirects, and inspect client-visible HTTP traffic and page content; **A2**, when a service places a secondary authentication gate after the URL, makes bounded guesses of user attributes used for authentication; and **A3** modifies the token or identifier of a valid URL when it is predictable or low-entropy, using ordinary HTTP requests to determine whether other users can be impacted.

Assumptions. We assume that the adversary has no legitimate credentials for the victim’s account, no privileged access to the service’s backend or databases, and no ability to compromise cryptographic primitives. We exclude phishing, social engineering, malware installation, carrier compromise, and denial-of-service attacks.

Goals. Under these capabilities and assumptions, we investigate whether exposure of an SMS-delivered URL can enable unauthorized disclosure or modification of user PII and whether the resulting impact can extend beyond the URL’s intended recipient.

3 Approach Overview

We aim to conduct a comprehensive investigation of SMS-delivered private URLs to understand how they expose user data, what weaknesses enable these exposures, and how their risks scale across services and users. Towards this goal, we build an end-to-end pipeline to investigate the risks while being reproducible and ethical.

Figure 1 depicts our end-to-end pipeline consisting of four stages: (1) Data Collection, (2) Preprocessing, (3) PII Detection and Validation, and (4) Analysis and Results. These stages are organized in the upcoming sections as follows:

Data Collection and Preprocessing (§4). We investigate the SMS-delivered URLs through the lens of public SMS gateways, an ethical vantage point consistent with prior studies [12, 15, 21, 41, 56, 57]. To this end, we begin by collecting messages from public SMS gateways, followed by extracting the URLs from the messages and capturing dynamic behavioral artifacts at the client-side (i.e., website UI, network logs, and HTML) for unique URLs (§4.1). We then remove duplicated UI captures and extract the embedded text using multilingual OCR on them, followed by structured extraction of JSON payloads from network logs and HTML (§4.2).

PII Detection and Validation (§5). We then use LLMs to flag URLs that expose PII in their artifacts (§5.1), followed by an expert validation (§5.2). To support effective detection, we construct a custom PII lexicon informed by prior literature and tailored to web-based observations. However, validating PII in UIs is considerably easier than in network logs due to the diversity in the number and size of JSON payloads. Therefore, we design a hierarchical LLM-based PII detection and expert-annotated validation pipeline. We first detect and validate PII in UIs, followed by detecting and validating PII in payloads (for the URLs with no PII in UI).

Analysis and Results (§6). Finally, we analyze the validated PII-exposing URLs to understand how exposure occurs, how it scales, and what risks it creates for users. We begin by examining the initial access paths and exposure lifetimes of SMS-delivered URLs,

focusing on whether URLs behave as bearer credentials and how long exposed links remain usable (§6.1). We then study whether a single exposed URL can lead to cascading leakage across other users of the same service by analyzing token predictability, entropy, and bounded mutation behavior (§6.2). Next, we examine authentication gates that rely on user attributes such as ZIP codes, dates of birth, or surnames, and evaluate whether these gates provide meaningful resistance against guessing attempts (§6.3). We further analyze the level of privilege granted by exposed URLs, distinguishing between read-only PII exposure, editable user records, and cases where URL possession grants account-level access (§6.4). To assess downstream risk, we examine how multiple co-located PII attributes can be combined into user profiles that may support targeted phishing, social engineering, identity theft, or financial abuse (§6.5). Finally, we look beyond the initially visible page to identify hidden data leaks caused by UI navigation and client-side overfetching, where additional PII is exposed through subsequent interactions, API responses, WebSockets, or HTML-embedded state (§6.6).

4 Data Collection and Preprocessing

SMS data collection is a crucial step to understanding the risks associated with SMS-delivered URLs. However, the SMS data contains irrelevant messages (e.g., OTPs), duplicate landing pages (e.g., redirects to the Instagram login page), and repeated authentication pages (e.g., pharmacies asking for date of birth before showing a prescription quote). Therefore, to limit our exposure to PIIs (further explained in section 4.2), the data must be preprocessed for subsequent analysis within our pipeline. Table 1 describes our resultant dataset corpus, emphasized in the rest of the paper.

Table 1: Data Collection and Preprocessing

Dataset Form	#URLs
Messages Collected	33,128,103
URLs Extracted	322,949
Unique Active Final URLs	147,251
Unique UIs	87,080
JSON Payloads	14,544

4.1 Data Collection

We examine SMS-delivered URLs through the lens of public SMS gateways, offering an ethical perspective into the SMS ecosystem. Our analysis begins by scraping these gateways to extract URLs and their associated dynamic attributes.

Collecting Messages from Public SMS Gateways. We compiled a list of public SMS gateways from prior work [12, 15, 21, 41, 56, 57]. However, we realize that most of these gateways are short-lived and no longer active. We therefore used search engines and advanced search queries (Google and DuckDuckGo) to identify active alternatives. We then filtered the ones that (1) do not require authentication, (2) are free to use, and (3) retain SMS messages for at least 24 hours. As a result, we identify 25 SMS gateways in total (complete list in appendix C). We developed custom scrapers using Playwright [4] for each gateway and collected all available messages by April 2024 (the

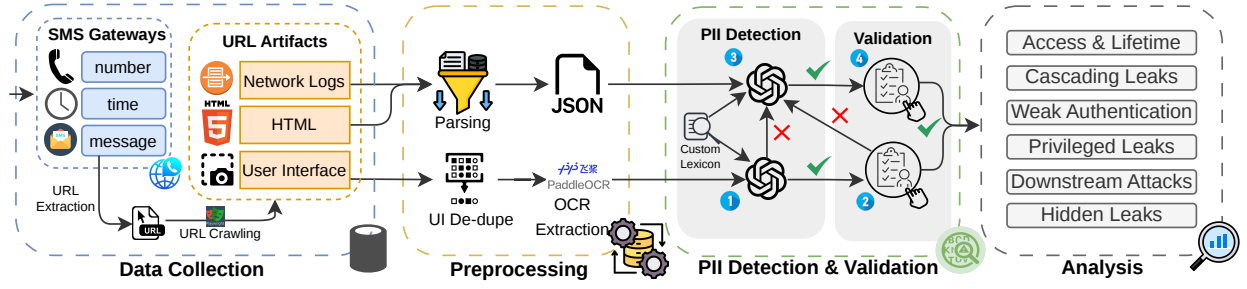


Figure 1: Our pipeline proceeds in four stages: (1) **Data Collection:** collect messages from public SMS gateways, extract URLs and crawl them using Playwright to capture Network Logs, HTML, and UI; (2) **Preprocessing:** run multilingual OCR on de-duplicated UI captures and extract JSON objects from Network Logs and HTML; (3) **PII Detection & Validation:** refine the PII lexicon and use an LLM to detect PII followed by manual validation; (4) **Analysis:** analyze initial access paths and exposure lifetimes, cascading leaks from user enumeration, weak authentication, privileged data leaks, downstream user-profile risks, and hidden leaks.

date of our data collection). To minimize interference, we enabled ad-blocking [1] to suppress advertisements. Additionally, in line with the ethical considerations employed by [54], we introduced a two-second delay between probe requests and avoided using concurrency. The messages collected, along with the receipt time, were stored in SQLite databases. In total, we collected 33,128,103 messages across 30,203 unique disposable numbers.

Extracting URLs From Messages. We find that gateways do not follow a consistent format when embedding URLs in messages, and some reformat message content by using line breaks, spaces, and wrapping around text. We account for the variations when extracting URLs from the messages. As a result, we identified 485,286 messages containing at least one valid URL, out of which, we extracted 322,949 *unique* URLs spanning 10,903 unique domains. Consistent with prior work [12], the URLs are predominantly OTP links as shown by four out of the top five domains: (t.me (21.88%), ig.me (9.07%), py.pl (8.65%), and viber.com (1.97%)). Moreover, 163,961 (50.77%) URLs in our dataset belong to domains ranked within the Tranco top 50k [34], highlighting that our dataset predominantly consists of prominent and widely used online services.

Extracting Dynamic Attributes. After extracting the URLs, we captured three URL attributes (UI captures, network logs, and HTML) to understand how each service handles SMS-delivered links, what authentication or authorization checks are applied, and where user data is exposed. We iteratively visit the URLs using Playwright [4] without multithreading and with a 2-second inter-request delay to avoid imposing excessive load on remote services, following web security measurement guidelines from prior studies [54]. Al-Fannah et al. [10] reported an average full-page load time of 19s across 10K websites and recommended that crawlers should not time out before at least 20s. Therefore, we wait for the networkidle state, with a 20 s timeout, before proceeding to the next URL to account for slow-loading pages and delayed network activity. We removed duplicate URLs previously, but unique URLs can lead to the same final URL due to redirects. Therefore, we capture the full-page UI screenshot (.png), network logs (.har), and HTML

snapshot (.html) only for unique final URLs to prevent redundant data collection.

Overall, we captured the attributes of 147,251 (45.60%) URLs from the initial set of 322,949 URLs. By collecting artifacts only for unique final redirected URLs, we stopped ourselves from performing recurring collection for 83,419 (25.83%) URLs, and 92,279 (28.57%) were inaccessible due to network timeouts or connection errors. One reason for the high inaccessibility rate is that we visited these URLs at least a year after the messages were received because we wanted to focus on long-lived PII exposing URLs and avoid ephemeral and one-time access links, including OTP flows.

4.2 Preprocessing

Data collection provides three raw artifacts per URL: UI captures, network logs (HAR), and HTML. However, some artifacts may correspond to short-lived URLs, leading to error pages such as 404 – *Not Found* or duplicates when different URLs produce the same website view. Therefore, preprocessing transforms the raw UI, HAR, and HTML artifacts into a deduplicated set of analyzable textual and structured representations suitable for scalable PII detection and validation. Preprocessing is performed at two levels: (1) UI, and (2) Network and HTML. The UI stage filters duplicate views caused by redirects or error pages and extracts text from captures using OCR. The Network and HTML stage focuses on extracting JSON payloads from network logs and HTML content. The output of this module is a cleaned, deduplicated dataset of UI text and JSON data, which facilitates the PII detection and Validation step (§ 5).

UI Pre-processing. Web services often generate URLs that redirect users to generic or repeated pages, such as Instagram login screens before displaying content, pharmacy date-of-birth prompts before showing prescription details, or shared error pages for expired or broken links. To address this, we only retain unique UI captures by removing byte-identical screenshots of the rendered URLs. We then locally run multilingual OCR on each retained UI to extract textual data from it. While HTML or DOM-based extraction is possible, robustly extracting only rendered, visible text from them at scale is challenging. First, an HTML-only analysis would miss

dynamic content that is loaded and rendered *dynamically*. Second, DOM-based analysis would miss rendered content outside the DOM (e.g., images, PDFs, canvases), which OCR reliably captures. OCR also captures text rendered outside standard DOM text nodes (e.g., images/canvas). We complement this UI-visible view with network-intercepted data (i.e., HTML, JSON) to analyze non-visible leaks.

De-duplication. We found that multiple URLs resolved to visually identical pages (e.g., shared error handlers). Our initial approach explored perceptual/structural similarity methods (e.g., SSIM, pHash, feature matching) because they can group pages that are visually similar despite minor rendering differences. However, we realized that they can over-merge pages where “*small*” visual differences encode sensitive identifiers (e.g., names, addresses, order IDs). Considering the loss of critical details, we resort to only removing byte-level screenshots by computing MD5 checksums of all UI captures and retaining only the first occurrence of each unique hash. As a result, we identified 87,080 (60.37%) unique UIs (3,958 domains) out of the 147,251. It is essential to note that this method does not discount near-duplicate UIs (e.g., distinct OTP codes). Therefore, our resultant dataset of unique UIs is still dominated by OTP-verification destinations, mainly `t.me` (72.69%) and `viber.com` (4.92%). The remaining entries in the top ten domains are global or custom URL shorteners that redirect to heterogeneous pages. In the deduplicated dataset corpus, we found that 76,524 (87.88%) URLs were ranked within the top 50k according to the Tranco list [34].

OCR Extraction. We take an LLM-based approach to identify PII in the UI. This necessitates extracting text from the UI captures, formatted as images. A reasonable option is utilizing the HTML to extract the text instead of extracting text from images. However, HTML-based text extraction has its own challenges, such as missing the contents rendered outside the DOM (we discuss this in detail in this subsection above under “UI Pre-processing”). Additionally, we use a text input for LLMs instead of passing UI captures directly to a vision or multimodal model because the target signals (PIIs) in our study are primarily textual identifiers. Furthermore, in the vision or multimodal model design, the LLM would need to both extract text from the screenshots and detect PII in that text. However, our current design relies on tools specialized to extract text from the images. Additionally, public SMS gateways are accessible around the globe, making the messages linguistically diverse and the webpages heterogeneous [55]. Therefore, we employ a robust multilingual OCR pipeline based on PaddleOCR 3.2.0 (PP-OCRv5, Universal Scene Text Recognition) [16] to extract embedded text from UI captures. In our deployment, PaddleOCR runs inference locally, ensuring that potential PII is not uploaded to external servers.

Network and HTML Preprocessing. Websites typically make API or WebSocket requests to render the UI and can have information embedded in the HTML that is invisible in the UI [50, 64]. Therefore, to conduct a comprehensive investigation of the client-side behavior, we focus below on extracting JSON data objects embedded in the HTML, as well as JSON responses from APIs and WebSockets. We limit our analysis to JSON payloads because they commonly contain user data [65], while substantially reducing the volume of data to be processed and validated. Together, the extracted JSON objects and OCR-derived UI text provide a normalized representation of visible and client-side data, allowing subsequent PII detection

Table 2: Custom PII lexicon creation. We retained 66 PII types from 287 policy-derived terms

Removal Reason	Examples	#Keywords
Repetition	(<i>location, geolocation</i>)	86
Generic/Vague	(<i>personal data, physical</i>)	75
Not Likely in Text	(<i>visual, thermal</i>)	40
Not Likely in Webpage	(<i>device identifier, beacons</i>)	18
OTP or Login Code	(<i>access code, security code</i>)	2
Total Removed		221

to be performed more efficiently by reducing redundant data and limiting analysis to relevant textual and structured content. While parsing the network logs (stored as HAR files), we select response bodies based on Content-Type. If a body is JSON, we parse it directly; if it is JSONP, we unwrap the function call and parse the payload. Similarly, if a body is HTML, we extract embedded JSON from (1) `<script>` tags whose type is `application/ld+json`, `application/json`, or `application/manifest+json`, (2) the widely used `__NEXT_DATA__` blob, and (3) inline assignments whose right-hand side is strict JSON (e.g., `window.* = { ... }`). We then perform JSON extraction over the saved HTML files. After canonicalizing and de-duplicating extracted JSONs using SHA-256 hash, we extracted 14,544 JSON payloads from 87,080 unique URLs.

5 PII Detection and Validation

The preprocessing module provides us with the text in the UIs and normalized JSON snippets (from Network Logs/HTML). Since preprocessing converts raw artifacts into unique and structured representations, the data is now suitable for LLM-based detection of potential PII exposures, followed by expert validation. Therefore, this section describes the PII detection and validation module.

Prior work has largely relied on the implementation of rule-based matching against raw strings [11, 25, 61]; however, this technique produces brittle results when multiple languages, templates, and layouts are involved. On the other hand, recent studies have explored using LLMs to identify PII [14, 53, 58] and the associated risks of hallucinations, leading to inconsistent labels. We therefore design a pipeline where an LLM flags potential PII exposures, and expert reviewers validate PII-exposing URLs.

Hierarchical Detection and Validation. The preprocessed artifacts contain two complementary views of each URL: UI text that approximates what a link holder can directly observe, and JSON payloads that may expose additional client-side data not rendered in the browser. We therefore use a hierarchical strategy to prioritize the lower-volume, more interpretable UI view first, and then analyze JSON payloads only where UI analysis does not already confirm an exposure. This ordering reduces redundant LLM processing and reviewer effort and improves validation reliability because PII in visible UI is easier for reviewers to inspect in context. After the UI-based validation, we run the JSON-based detection LLM on data that (1) the UI-based LLM flagged as non-PII or (2) was discarded during the manual UI-based validation. Finally, we also validated the JSON payloads flagged by LLM.

PII Taxonomy. Before we attempt to use LLMs to identify PII, it is essential that we ground our definition of PII on the foundations of the extensive research in this space. While the privacy domain has resulted in a multifaceted definition of PII, covering various use cases (e.g., biometric information, health records, advertisement interactions), it is important that we reduce the definitions to a narrower web domain, enabling efficient LLM adoption by saving on token usage. We utilize the taxonomy created by Lovato et al. [37] as a baseline because they provide a comprehensive and privacy policy-derived lexicon of PII types. For the 287 keywords in the original lexicon, we removed 221 based on the following filtering strategy: (1) repetition: When a PII type is similar to another, keep only the most clear and/or standard term, (2) too generic/vague: When a PII type is overly generic or vague and does not meaningfully represent identifiable information, (3) not likely in text: When a PII type can not be directly identified from text, (4) not likely in webpage: When a PII type is not likely to be present in or verified just from the website UI or JSON payloads, or (5) otp or login code: When a PII type is related to OTPs or Login Codes. Consequently, we retain 66 unique PII types relevant to webpage UI and JSON artifacts (see Appendix §D). The most frequent removal cause was repetition (86 terms, 38.91%), followed by overly generic or vague descriptors (75 terms, 33.94%). Table 2 summarizes the removal reasons for the 221 PII types in the original PII lexicon.

5.1 PII Detection

This subsection focuses on detecting PII in the textual data from the preprocessing stage. However, given the volume and complexity (of OCR text & JSON payloads), we need a scalable detection pipeline. Traditional methods like rigid pattern matching miss context and provide unreliable results. However, LLMs have been shown to be effective for privacy-relevant text analysis because they can capture semantic context beyond rigid patterns and support flexibility across heterogeneous inputs. For instance, Liu et al. [35] show that LLM-based PII extraction outperforms expressions, keyword search, and entity detection.

A rule-based alternative would require maintaining hand-crafted detection heuristics for 66 PII types across multilingual text, heterogeneous webpage templates, and structurally diverse JSON payloads. More importantly, our task required contextual understanding. For example, a name may identify the user, appear as public company information, or occur as unrelated content on the page. We therefore use the LLM as a scalable candidate-detection stage followed by manual validation.

We constrain our LLM to classify PII based on our improved set of PII identifiers with a strict output schema. For each input, the LLM first determines whether PII is present and, if so, labels the exposure with one or more taxonomy-defined PII types, such as first name, phone number, or email address. We apply a zero-shot LLM pass over OCR text for UI-based detection and JSON objects for the JSON-based detection. Inference runs locally via LM Studio [6] using the open-source gpt-oss-120b model, which has been shown to match the performance of GPT-4o-mini [9]. This setup ensures that no content is sent to third-party services and that all data remains within the analysis environment. We construct

a *system prompt* that sets high reasoning and enforces an exact output schema, and a *user prompt* that includes the reduced PII lexicon and the OCR text or JSON payload.

5.2 PII Validation

LLMs are known to hallucinate, making it difficult to formulate conclusive remarks. Therefore, it is essential that we include a human validation step to mitigate false positives and label drift [28]. For us, validation ensures that data leaks correspond to real, user-linked PII and allows us to categorize their type.

To ensure label correctness and reduce individual bias, two expert annotators independently review all LLM-flagged items, first for UI and then for JSON payloads. Reviewers label each item *true positive* (PII present) or *false positive* (no PII) and correct the categories when needed. Reviewers applied the following criteria when labeling something as *false positive*: (1) PII-asking page: asking to provide PII in forms, (2) sample/demo content: prefilled with sample content like example.com, ads with sample content, or testing dummy data in dev sites, (3) public or company information: website's phone number, email, address or anything that is intended to be public like names on social media, reviews with users' names, or disposable phone numbers, (4) keyword match: Exact or similar keyword match for PII like Name for name or SSN for social security, or (5) misclassification: it is a wrong match, mostly due to partial info like (user*****@gmail.com).

After labeling by the two expert annotators, we resolved disagreements by discussion and joint review. In case of no agreement after discussion, **we label it as a false positive to avoid over-estimation.** After conflict resolution, we obtain the final dataset containing labels and categories for further analysis. We use the following methodology for UI and JSON-payload validation.

UI Validation. We opened the UI capture for each of the flagged URLs and looked for visible user PII. We fix the PII types if the LLM over-labeled or under-labeled any true positive. If the LLM flagged a UI as having PII but we do not see any user PII (due to any of the reasons described earlier), we label it as a false positive.

JSON Payload Validation. Similar to the UI validation, we begin by analyzing the raw LLM output. However, considering the size and complexity of the payloads, we specifically focus on the identified PII. We open the JSON snippets for the URL and search for the PII; otherwise, we flag the sample as false. We do so because of the time required for the comprehensive analysis of every payload. We skim through the parts around that keyword to understand the context. For example, Fig. 8 (in appendix E) shows that the exposed email appears within the order object and alongside order-specific fields. This context confirms that the email is associated with the user's order rather than belonging to a developer, company employee, or unrelated log entry. Based on this information, we only label something as a true positive if we are certain that the exposed PII is the user's private information.

5.3 Detection and Validation Results

UI Results. Our LLM detection over OCR text for all 87,080 unique UIs flagged 1,916 (2.20%) candidates for manual validation. Reviewer one labeled the 538 *true positives* while reviewer two labeled 677

Table 3: PII Detection and Validation Metadata

Dataset Form	#URLs	#Services
Initial Set of URLs	87,080	3,958
PII Detected in UIs	1,916	819
PII Validated in UIs	553	121
PII Detected in JSON payloads	1,958	844
PII Validated in JSON payloads	148	67

true positives. After conflict resolution, we confirmed 553 *true positives* (PII present) and labeled 1,363 as *false positives*. The common cause of discrepancies in this step was that one reviewer treated the exposure of the SMS gateway number as user PII; however, a consensus was reached that it should not be labeled as PII, as this number is already compromised or known to the adversary.

JSON Payloads Results. We found that 14,544 URLs had JSON payloads in their network logs or HTML, 440 of which were marked as having PII during UI validation. Our LLM detection over JSON payloads for the 14,104 payloads flagged 1,958 (13.88%) candidates for manual validation. Reviewer one labeled the 163 *true positives* while reviewer two labeled 161 *true positives*. After conflict resolution, we confirmed 148 *true positives* (PII present) and labeled 1,810 as *false positives*. The main source of false positives at this stage was the LLM’s tendency to flag public information, such as company contact details, as PII, even though the prompt explicitly instructed the model to ignore such content. During this step, the reviewers had comparatively fewer discrepancies due to experience from the UI pass. The only conflicts arose when reviewers could not tell from the JSON payloads whether the PII under investigation was relevant to the user. For example, we did not label a non-company email address as user PII unless nearby JSON fields provided concrete evidence linking it to the user.

Validated PII Exposing URLs. In total, we validated 701 URL endpoints across 177 services. Table 3 summarizes the results from detection and validation. The high false-positive rate primarily reflects the difficulty of distinguishing user-linked PII from syntactically similar but non-user information. For example, many flagged values were genuine names, email addresses, or postal addresses, but belonged to developers, companies, or other public entities rather than the user associated with the URL. During manual validation, we confirmed that none of the 177 affected services were scams. We also found that 55 of the services appear in the tranco top 1M list, including 14 ranked within the top 50K.

6 Analysis and Results

This section describes the analyses of the validated PII leaks and our findings. Specifically, we examine the initial access paths and exposure lifetimes of leaked URLs (§6.1), cascading leakage through user enumeration (§6.2), weak authentication secrets (§6.3), privileged account access (§6.4), potential downstream attacks (§6.5), and hidden data leaks caused by client-side over-fetching due to improper server-side filtering (§6.6). Table 4 summarizes the analyses, OWASP/CWE mappings, and key results across §6.1–§6.6.

Whenever necessary, we explain our findings and emphasize the root cause with an example vulnerable service. It is essential to note

that we conduct the below analysis under strict ethical guidelines as detailed in Appendix B. Additionally, we only name the service if they have already fixed the vulnerability.

6.1 Initial Access and Lifetime of URLs

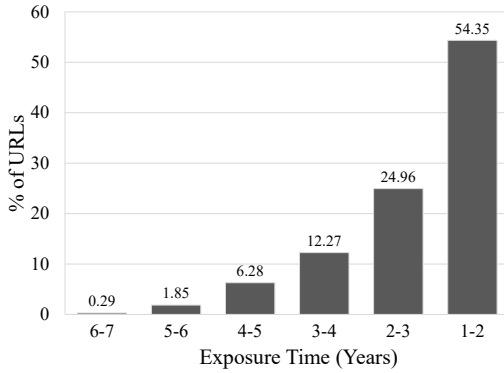
SMS-delivered private URLs are intended to provide temporary, low-friction access, but they become risky when possession of the link is sufficient to retrieve user PII. This risk depends on both how the service makes the private webpage accessible to the non-intended user (aka adversary) and how long the URL remains active. The exposure path reveals the root cause of the leak, while the URL lifetime determines how long the link can be logged, scraped, forwarded, leaked, or rediscovered.

Methodology. We open the URL with DevTools enabled to observe redirect links, network logs, and headers to pinpoint the initial source of PII exposure. We identify the exposed PII using the search feature under the Network tab in DevTools. The text search gives us item(s) that has the PII and then we manually step through them in reverse chronological order to locate the first item that contained PII (or the item that requests it). During the process, we check the “Headers” and “Response” tabs to analyze each request/response chain. In case of tokenized URLs, we examine API calls and WebSocket messages for token usage. If a token is used in the request header but no API or socket activity is visible, we classify the exposure as **server-rendered**. If we find the PII in the original URL or the redirect chain headers, we mark it as **URL-embedded PII**. We also measure exposure lifetime, as having long or permanent access URLs significantly increases the probability of exploitation by an attacker. For this reason, sensitive URLs must be only accessible for a short and defined time [48]. We measure the exposure time as the difference between the SMS delivery date and the URL crawling date (**July 26, 2025**). Because all validated URLs were still active on the crawl date, the exposure time we report is a lower bound on their active duration. It is essential to remember that even if a domain were re-registered by a different entity between the time the message was sent and our crawl, it is extremely unlikely that the same access token would be reused. To further understand the PII exposure behavior, we categorize the services (domains) by their associated industries. Similar to prior work [69], we utilize the VirusTotal Forcepoint [20] classification engine to obtain the industry for each domain. In total, the Forcepoint engine was able to classify 118/150 domains whose data was extractable using VirusTotal API; the list of categories is presented in Table 5.

Results. Tokenized bearer links were the dominant root cause, totaling 172/177 (97.18%) services. Of these, 92 (53.49%) called an API, 79 (45.93%) were server-rendered, and 1 (0.58%) used a WebSocket. In the remaining 5/177 (2.82%) cases, PII appeared directly in an original URL or redirect chain. As shown in Figure 2, we observe that 54.35% of PII-exposing URLs were sent within a period of 1 to 2 years, and the remaining 45.65% were at least two years old. The drop in exposure time from 1-2 to 2-3 years suggests that some URLs may expire after 2 years, but we believe it is because of dataset bias as only a handful of public SMS gateways store SMS data for longer than 2 years. Still, we see PII-exposing URLs leading back to 2019. This indicates the presence of long-lived URLs, and when combined with PII exposure, it leaves the user’s PII at a higher risk

Table 4: PII analysis results and root-cause mapping with OWASP/API Top-10 and CWE

Analysis Category	OWASP / API Top-10	CWE	Key Result
Access & Lifetime §6.1	A01:2025; API2:2023	CWE-284; CWE-287; CWE-862	Token (Server: 79; API: 92; Socket: 1), URL: 5
Cascading Leaks §6.2	A01:2025; A04:2025; API1:2023	CWE-331; CWE-307; CWE-425	Low Entropy: 125, Enumerable: 13
Weak Authentication §6.3	A06:2025; A07:2025	CWE-287; CWE-307	Weak Authentication: 4
Privileged Leaks §6.4	A01:2025; A06:2025; A07:2025	CWE-359; CWE-862; CWE-613	Editable: 14, Account Access: 5, Both: 1
Downstream Attacks §6.5	A01:2025	CWE-200; CWE-359	User Profiles: 206
Hidden Leaks §6.6	API1:2023; A06:2025	CWE-201; CWE-425; CWE-862	Overfetching: 76, UI Interactions: 8

**Figure 2: PII-Exposing URLs by Exposure Time (n=701)**

of attack. Additionally, we find that the most common categories are “business” and “IT” with 27 and 23 services, respectively. Surprisingly, **19 services belong to financial services**, contrasting with the stricter expected levels of security regulations in the financial industry [8]. Table 5 shows popular categories for 105 of the 118 service categories classified by Forcepoint. Various security regulations such as GDPR [7] and guidelines like NIST SP 800-122 [44] advocate for secure data retention policies. Although NIST clearly states that PII should not be stored for long periods, they lack specifying explicit time-frame to enforce provider compliance and allow PII retention when it is “strictly” necessary.

Table 5: Industry categorization of service providers (domains) using Forcepoint classification engine

Industry	#Services	Industry	#Services
Business	27	Job Search	4
IT	23	Real Estate	4
Finance	19	Educational	3
Shopping	9	Business App.	2
Vehicles	6	Restaurants	2
Health	4	Media	2

Impact. This “URL-as-credential” antipattern amplifies risk in SMS workflows where messages are routinely logged or intercepted. Operationally, these patterns treat the URL itself as the authenticator. The information about the source of the data leak helps guide further analysis and provide informed recommendations to the security community. We also found that approximately 46% of URLs remained active after 2 years, significantly increasing the risk of attacks for these users. We argue that URLs exposing or granting access to PII should have short, explicitly defined lifetimes based on the sensitivity of the exposed data and the expected user workflow, and that access to those records should remain restricted behind strong authentication gates. These gaps are particularly concerning for critical sectors such as “IT” and “Finance”, which are considered critical infrastructure [17] (highlighted in Table 5). URLs from “Finance” mostly led to pre-filled loan applications, which could reveal users’ financial status [39], making them vulnerable targets for scammers.

Takeaway: Tokenized bearer links account for $\approx 97\%$ of observed PII exposures, and 45.65% of validated PII-exposing URLs remained active for more than two years. SMS-delivered private URLs should not act as standalone authenticators: they should be short-lived, one-time use where possible, protected by server-side authorization checks, and governed by clear industry-specific lifetime limits.

6.2 Cascading Leaks from User Enumeration

A leaked SMS-delivered URL can expose the intended recipient’s data. However, predictable identifiers within URLs can expand that harm to other users of the service. In other words, if tokens are low-entropy or derived from predictable sequential identifiers, an adversary who has access to only one valid URL would be able to guess neighboring URLs and access records of the entire user base.

Methodology. Due to the higher percentage of observer URLs being tokenized bearer URLs, we analyze their token entropy to assess whether these tokens are enumerable. We follow the theoretical foundations laid by MITRE’s Common Weakness Enumeration, specifically CWE-331: Insufficient Entropy [40], and OWASP [49] guidelines. For every token identified during root-cause analysis, we estimated its entropy as $H = \log_2(|\mathcal{A}|^L)$, where L is the token length and $|\mathcal{A}|$ is the effective alphabet size. We experimentally verified whether case sensitivity contributed to entropy by flipping the first alphabetic character in the URL and if the response was the same as the UI of the original URL, the alphabet’s possible values were reduced from 52 to 26. If the token only contained hex

characters (0-9 and A-F), we assumed it is a hex token (16 possible values per character) and not alphanumeric character (36 possible values per character). Tokens with $H < 64$ bits were flagged as potentially enumerable. Further, we manually attempted up to 10 token mutations for every tokenized URL and stopped at the first hit. We then classified a domain as enumerable if a modified URL yielded another user's PII.

Results. Among the 172 tokenized URL services (recall, we select one representative URL per final domain), 125 (72.67%) services have a token with entropy below the 64-bit threshold and were flagged as potentially enumerable. During the 10 manual token mutations, we obtained 13 successful hits (10.40%). In these cases, we confirmed only the existence of exposure and did not store screenshots, network logs, or PII due to ethical concerns, since these URLs were not directly retrieved from public SMS gateways. Apart from entropy, enumeration also depends on the density of valid users within the token space. However, our study does not estimate this density or attempt to confirm large-scale enumerability, because doing so would require automated token mutation and requests at a scale that would be ethically inappropriate.

Example (Anonymous Transport Company): It is an auto transport company working across 49 states. We find that quotes assigned to users have a unique token assigned, formatted as 123456-AB. However, the pattern was easy to identify: changing the token (which is also the order number) to 123457-AB resulted in access to the next user's quote. Further, the URL in our dataset had the order number around 650K (the 123456 in the example URL), but checking an order number in the range of around 300K resulted in another user's quote. Therefore, it shows that linear scanning may reveal at least 650K users' PII. Also, this issue was not unique to this service: four other services among the 13 confirmed enumerable cases made similar design choices.

Impact. Low-entropy or predictable tokens turn a single leaked URL into a horizontal compromise. Once an adversary learns the token format, they can probe adjacent or otherwise guessable identifiers using ordinary HTTP requests, potentially discovering additional users' records without compromising the provider's infrastructure. The risk increases when many valid records occupy a small or sequential token space: each successful guess requires less time and effort due to the saturated token space. Consequently, even users whose SMS messages were never exposed can be affected if the service uses predictable SMS-link identifiers.

Takeaway: We found that 125 services use tokens with estimated entropy below 64 bits and are therefore potentially enumerable. For 13 of those services, bounded testing confirmed access to another user's data within the first 10 mutations. Service providers should use cryptographically random, URL-safe tokens with at least 64 bits of entropy.

6.3 Weak Authentication Secrets

For a majority (97.18%) of the services vulnerable to data leaks, we find that the adversary can directly get access to user data. However, we also find that services in our dataset request authentication secrets, such as an OTP or date of birth, before exposing user data. As intended, we assume only users know this secret and consider social engineering-based attacks out of scope. It is also expected

that services implement a limit on how many times a user is allowed to guess the authentication secret. Not doing so creates an avenue for typical brute force attacks. Below, we describe our approach to uncovering these potential attacks and present our findings.

Methodology. For pages requiring authentication before granting access, we check whether the required value appears in our SMS corpus for that number or is trivially guessable (e.g., ZIP code). We also inspect network logs for clues that could reduce the cost of enumeration for an attacker. When encountering URLs with such authentication constraints, we perform up to ten automated attempts, stopping immediately upon a successful match, exhaustion of attempts, or any indication that further attempts are blocked. During this process, we observe server responses and timing behaviors to identify lockouts, CAPTCHAs, or escalating delays.

Results. Among the 177 services, five services show limited PII in the initial view but require authentication to access more privileged information such as health records. We find that four of these services implemented weak authentication parameters. These services were also vulnerable to weak or absent rate-limiting as we were not blocked during the 10 automated attempts.

Example (Anonymous Financial Company): It is a debt purchase company that has helped over 2 million customers. As a well-known financial company, it protects users' private information by requiring two authentication secrets before showing sensitive data. Unlike the other two of the four cases in our dataset, which required one secret, this appears more secure. However, by searching the SMS data for the same number using the given first name as a keyword, we were able to find one of the authentication secrets, i.e., the surname. Consequently, we were left with only one enumerable secret (date of birth) which was not protected by rate limiting.

Impact. Absence of throttling allows the authentication to collapse under trivial enumeration and become guessable credentials, turning a leaked link from a read-only privacy breach into a pathway for account takeover or broader PII exposure. Additionally, when combined with predictable tokens, this can create a false sense of security while still enabling a service-wide cascading data leak.

Takeaway: We identify 4 services with enumerable authentication. This underscores the need for strict limits on the number of authentication attempts to prevent easy guessing of secrets such as last name, ZIP code, or date of birth.

6.4 Privileged Data Leaks

In the previous subsections, we describe the risks of private data being exposed to an adversary. However, the impact can be more severe when possession of the URL grants privileged access. This could be in form of edit access to user records and/or logged-in access to user's account. Such exposures escalate the impact from confidentiality loss to integrity risk.

Methodology. To determine the exposure level, we analyze only the first loaded page's UI to check if the exposed PII is editable or read-only from the adversary's perspective. If a URL lands the adversary on a page containing PII that they can edit, we consider it as *editable*. Otherwise, if the PII is not editable or available in the JSON payloads, the exposure is termed as *read-only*. We also confirm whether the endpoint is in a logged-in state by checking

whether the rendered view has authenticated-only navigation elements (e.g., “My Account,” “Billing,” “Logout”) or personalized headers (“Welcome, <name>”), or pages that enumerate private resources (applications, orders, messages).

Results. The majority of cases (162/177) were read-only data spread across UI (103/162) and JSON payloads (59/162). The remaining (15/177) editable data instances were pre-filled forms in the UI. Beyond editable forms, we identified six services that granted account access based solely on possession of the URL. One of the account-access cases overlapped with the editable cases. Therefore, we found 20 unique services with privileged access.

Example (iHire): It is an employment platform that brings candidates and employers together in 57 industry-focused communities such as iHireDental, iHireConstruction, and iHireChefs. They are used by 103K organizations (e.g., Walt Disney World, Marriott, Lowe’s) and have “3.45+ million active members in the last 30 days” [5], implying broad impact. They use SMS and email to send job application updates, including links with short tokens (five-character alphanumeric). These links redirect anyone to a page with “Welcome back, FirstName” and a “Continue” button underneath it (refer to appendix Fig. 5). Clicking on the button redirects to the logged-in profile with edit (and read) access to resume, contact details, address, job applications, and email & password for the account. This was also the only account-access case that overlapped with the editable cases, enabling account takeover by allowing the password and email address associated with the account to be changed without any authentication. It is also concerning because low token entropy, combined with a large user base, resulted in a saturated token space. Consequently, we were able to access another user’s account only after two token mutations, which once again reiterates the cascading effect described in §6.2.

Impact. These flows collapse the boundary between “viewing a page” and “acting as the user” of the service. *Read-only* exposure enables immediate harvesting of private data with commodity crawlers, while *editable* exposure or *account access* escalates the harm from confidentiality loss to integrity risk: adversaries can alter user records, submit incorrect information, change account details, or perform actions as the victim. This risk is amplified when these URLs are transmitted over unencrypted channels such as SMS.

Takeaway: We found that 14 services allowed adversaries to modify users’ PII, five services that grant account access, and one that gives both account access and edit access. Services should not show any PII without authentication, but they should be especially careful in the case of privileged access.

6.5 Potential Downstream Attacks

While individual PII attributes can be harmful, their risk is exemplified when multiple identifiers appear together. Prior work has shown the ability of a group of PII attributes to facilitate downstream attacks, such as spear phishing, social engineering, identity theft, and cross-dataset linkage. For instance, combinations such as name, phone number, email address, postal address, date of birth, financial identifiers, or government IDs can form actionable user

profiles. Such contextualized user profiles have great value for targeted marketing and financial fraud. Below, we explore the potential use of the PII leaks in actionable downstream attack pipelines.

Methodology. We look at the UI captures and JSON payloads for each validated URL to extract user PII. Our ground rules are that the URL should expose a user’s name (an established and effective identifier) *and* at least one type of contact PII: (1) telephone number, (2) email address, or (3) postal address. If a URL passes these conditions, we extract all visible types of PII. To avoid repetition, we concatenate all visible types of PII for each URL to construct a “user profile” and only unique profiles are kept to analyze what types of PII appear together and how some combinations can be more effective than others. Although all pages were publicly reachable, we treat the content as sensitive: plaintext PII values are deleted after analysis and only aggregate counts are stored.

Results. Among the validated set of URLs, 290 (41.37%) met our ground conditions. After filtering unique “user profiles”, we were left with 206 (29.39%) candidates. The broadest PII exposure (large user profile) came from a pre-filled loan application exposing first name, last name, email address, telephone number, date of birth, marital status, postal address, bank account number, state identification card number, and family (refer to appendix Fig. 3). Another notable combination was also from a loan application exposing first name, last name, email address, telephone number, postal address, bank account number (including routing number), and social security number in the network logs (refer to appendix Fig. 6).

Impact. The ability of adversaries to aggregate multiple PII attributes (e.g., date of birth, social security number, name) to *profile users* creates avenues for downstream attacks such as spear phishing, social engineering attacks, reconstruction attacks, and identity theft [18, 19, 47, 63, 68]. These profiles, even those passing our two ground conditions, can be leveraged to launch downstream threats. For instance, compound profiles with attributes such as *name*, *email*, *phone* enable high-confidence spear-phishing and cross-dataset linkage. Similarly, including *DOB* or *account identifiers* intensifies susceptibility to financial risks.

Takeaway: We identified 206 distinct user profiles, demonstrating how SMS-delivered URLs can facilitate social engineering and enable critical downstream attacks, including spear-phishing and financial impacts.

6.6 Hidden Data Leaks

The first page loaded from an SMS-delivered URL may not reveal the full extent of exposed data. Some services initially display limited information but expose additional PII after simple UI interactions, such as clicking buttons, opening tabs, expanding sections, or navigating to related pages. In other cases, the UI might have limited or no PII but underlying network requests made by the browser contain additional PII. In many cases, API responses may include additional fields or payloads that are not rendered, yet still remain accessible through browser developer tools (e.g., network logs). Therefore, we interact with the UI and analyze network logs to identify “hidden” leaks where the initial page appears to minimize or hide PII, but additional user data is exposed through subsequent navigation or client-side payloads.

Methodology. We examine webpages for navigational elements such as menus, buttons, pagination controls, and expandable accordions. We interact with these elements to uncover additional user PII, avoiding any state-changing actions such as form submissions. Upon the first occurrence of additional PII, we stop and label it as a “hit” for UI navigation. We also investigate whether JSON payloads from HTML and network logs contain additional user PII that is not exposed in the UI, since modern web applications often retrieve data through API responses, JSON payloads, or HTML-bootstrapped state before deciding what to display. To do so, we use the browser profile with DevTools enabled and navigate to the Network tab. For each PII string visible in the UI, we use Network search (body+headers) to find its originating response. Then, we manually skim these responses to find additional user PII not present in the UI. In addition, we record common patterns and root causes of misconfigurations that lead to overfetching of PII.

Results. We identify hidden data leaks in 84 cases where additional PII is exposed beyond what is visible on the initial page load. Specifically, we identify eight instances where additional PII is exposed through interaction and navigation on the webpage. On the other hand, we observed overfetching in 76 cases due to improper server-side filtering: network responses or server-rendered bootstraps carried more PII than displayed in the UI. Although technologies such as GraphQL support precise data retrieval, we identify three services that use it incorrectly, negating its intended security benefits. Instead of requesting only the fields required for the current view, these services retrieve complete user profiles or order details and then filter the response on the client side.

Case Study 1 (EverQuote): It allows users to shop for insurance across 160 insurance companies including top providers such as Progressive and Liberty Mutual Insurance. It is consistently ranked around 60k on the Tranco list and the company reports handling “35 million quote requests from consumers” [3]. One of the URLs in our dataset redirected to an insurance quote page from Everquote. It only showed (“Congratulations, FirstName”) with a *Show Quotes* call-to-action (refer to appendix Fig. 4). However, clicking on the call-to-action button led to a fully editable, pre-filled insurance quote form containing sensitive PII—full name, email address, gender, date of birth, and credit score range. Further, the URL had a short path (token), consisting of five alphabetic characters, which, combined with the large number of users, resulted in a saturated token space. Consequently, we were able to access another user’s quote only after three token mutations which also reiterates the cascading effect described in §6.2.

Case Study 2 (DiMOrder): It is a POS system for restaurants powering over 1,000 restaurants across Asia. It has received multiple awards including Forbes Asia 100 to Watch and is partnered with major services including Stripe [2]. Therefore, it is a trusted platform used for orders by restaurants. The URL in our dataset showed an order page in the UI with only the user’s address for delivery, which was the only information intended to be shown. Still, the JSON response from the API returned the entire order data: seller and user metadata, geolocation, contact details, order statuses, and even the driver’s license plate number (refer to appendix Fig. 7).

Impact. Although some developers may assume that “hidden in the UI” means hidden from attackers, this PII is still accessible

from static resources, making it vulnerable to client-side observers. Hidden data leaks expand the risk of SMS-delivered URLs beyond what is visible on the initial page, allowing anyone with the link to uncover additional user information through simple UI interaction or client-side inspection. Even when a landing page appears benign, UI interactions (often in just a few clicks) can reveal additional user PII that was not initially visible. This expands the leak from a single read-only snapshot into an exploratory path accessible to anyone with the link. Similarly, even if the visible page redacts or omits PII, excess payloads in client-side responses and HTML can silently leak sensitive PII. It is also important to note that, in many cases, service providers assume their endpoints expose minimal PII, while hidden UI paths and network logs reveal much more to an attacker.

Takeaway: We found that 84 services expose additional PII beyond the initial page, including 76 through client-side overfetching and 8 through UI interactions or navigation. This is a reminder to protect sensitive UI paths when accessing from an SMS-delivered URL. Further, pages should request only the PII fields needed by the current UI view. During development and testing, server-rendered bootstraps, API/WebSocket responses, and navigable UI paths should be audited for sensitive data.

7 Responsible Disclosure

For each vulnerable service, we manually reviewed its website to identify explicit guidelines for reporting security and privacy weaknesses. During this process, we found disclosure details in privacy policies or custom responsible-disclosure guidelines, while some services declared no disclosure pathway. Our experience highlights the need for organizations to maintain clear reporting instructions, dedicated disclosure channels, and sufficient resources to ensure the timely evaluation and resolution of security reports. The rest of the section describes our disclosure methodology and insights from the experience. Lastly, we describe the impact of our disclosures.

Table 6: Responsible Disclosure Status.

Disclosure Status	#Domains
Unreachable	27
No Response	132
Responded	11
Fixed	7
Total	177

Disclosure Methodology. We found that only five services included a dedicated section describing their security policy, and only two of these provided a bug-reporting page. In addition, seven services mentioned privacy-related reporting mechanisms in their privacy policies. For the remaining 165 services, we relied on any publicly available contact channels we could identify. We prioritized email addresses, followed by telephone numbers and web-based contact forms when available. In most cases, the only reachable channels were general customer-service email addresses or phone numbers, rather than contacts for dedicated engineering, security, or privacy teams. For 27 services, we were unable to identify any

viable public contact information. These observations offer insight into the broader security and privacy maturity of the ecosystem. Notably, one service with an explicit security section provided a detailed disclosure form accompanied by a PGP key; however, the key had expired nearly three years ago. In one case, involving a major U.S. sporting goods retailer, the customer-service team did not appear to understand the issue. We spent approximately 15 minutes on the phone explaining the vulnerability, first to a representative and then to a manager. They stated that they would follow up, but that never happened and the vulnerability remains unresolved.

Disclosure Insights. We find that responsible disclosure is often a labor-intensive process with a low likelihood of acknowledgment or remediation. Table 6 describes the disclosure status. Of the 150 reports we submitted, only 18 received responses, and just 7 out of those resulted in implemented fixes. To improve engagement, we adopted an alternative strategy by conducting telephone-based disclosures for services that had neither responded to email nor acted within 30 days. This approach enabled us to reach teams at four services, three of whom subsequently acknowledged the issues. One notable case involved a provider stating that they were “overwhelmed with the volume of scam bug reports and had assumed our disclosure email was also a scam.” It is important to emphasize that our reporting process adheres to strict standards: each email includes detailed reproduction steps and an accompanying video demonstration or relevant data.

Disclosure Impact. We estimate the scale using publicly reported active-user counts, where available, for fixed services that had potentially enumerable tokens. We estimate that the resulting remediations reduced exposure risk for at least 120 million users.

8 Related Work

SMS Abuses. Recent studies have focused on understanding security threats originating from SMS messages. Several works show that SMS metadata and timing patterns can be exploited to infer user locations [26, 27, 32]. For example, Bitsikas et al. [13] demonstrated that an attacker can deduce the location of an SMS recipient by comparing message-delivery timings against pre-collected measurements from receiver locations, and argued that mitigating this class of attacks requires complex architectural changes on the SMS infrastructure. Other studies analyzed attacks that use SMS messages as a bridge for spam and phishing campaigns [36, 42, 52, 56, 62, 66]. In this regard, Nahapetyan et al. [43] collected a longitudinal dataset of phishing SMS messages and categorized them into over 35K phishing campaigns, providing detailed insights into attacker tactics and countermeasures. Similar to our work, many of these studies utilized public SMS gateways as a way to understand real-world behavior of the SMS messages and their security implications [15, 41, 43, 55, 57]. Our work differs in that we focus on benign, service-generated messages containing private, user-specific URLs, and we analyze the design of these URLs and how they can expose user PII.

URL Security and Privacy. Researchers have put effort into discovering potential vulnerabilities, misconfigurations, and leaks in URLs [31, 38, 59, 71]. For instance, Matic et al. [38] trained a classifier to identify URLs pointing to sensitive user information. Over a database of 155 million sensitive URLs, the authors found questionable practices such as the usage of unencrypted protocols to transfer

sensitive data (HTTP). Moreover, Khodayari et al. [29] analyzed open redirect vulnerabilities and found 623 sites exposed to this issue. The authors revealed that this vulnerability is widespread and affects 9.7% of the top 10K websites. Similarly, Wang et al. [70] found overly excessive permissions in cloud-based API endpoints, potentially leading to data attacks. This study was performed over 1.3 million cloud apps, where $\approx 27\%$ were exploitable. Complementing these URL-centric studies, Rahaman et al. [51] evaluated PCI DSS certification for e-commerce websites, revealing substantial gaps between prescribed web-security requirements and their real-world enforcement. Prior work has also shown that low-entropy object identifiers and shortened URLs can permit enumeration and unauthorized discovery of resources [23, 45, 46]. Our work does not claim that address-space enumeration itself is novel. Instead, we contextualize a novel attack surface (public SMS gateway) that enables the accurate identification of target services and their token format without relying on blind discovery.

9 Conclusion

Service providers increasingly prioritize frictionless user experiences by adopting technologies that streamline access to their platforms. SMS remains a widely used channel for delivering private, single-click access URLs. However, the inherent insecurity of SMS has led to data leaks that can expose these URLs to unauthorized adversaries. This paper presents a comprehensive examination of the security and privacy implications arising from such unauthorized access to SMS-delivered URLs. Using public SMS gateways as a lens, we analyzed 322K URLs extracted from more than 33 million SMS messages, ultimately identifying expert-validated PII leaks across 701 URLs associated with 177 services. We found 125 services with low-entropy tokens that were potentially enumerable and confirmed enumeration in 13 services under bounded testing (10 manual attempts), demonstrating the potential for exposure to extend beyond the intended recipient. Additionally, we identified 20 services that grant adversaries privileged access, 84 services that expose additional PII beyond the landing page, and 206 unique user profiles that could be exploited for downstream attacks such as spear-phishing. We recommend three concrete defenses: (i) URLs delivered over SMS should never act as standalone authenticators; (ii) tokens should provide ≥ 64 bits of entropy from a cryptographically secure RNG; and (iii) services should never expose more PII than what is needed on the landing page and server-rendered bootstraps, API/WebSocket responses, and navigable UI paths should be audited for sensitive data.

References

- [1] [n. d.]. @cliqz/adblocker-playwright — cliqz. <https://www.npmjs.com/package/@cliqz/adblocker-playwright>. [Accessed 06-02-2026].
- [2] [n. d.]. DimPOS - One-stop shop POS System — dimorder.com. <https://www.dimorder.com/en/>. [Accessed 10-10-2025].
- [3] [n. d.]. FAQ | EverQuote — everquote.com. <https://www.everquote.com/faq/>. [Accessed 10-10-2025].
- [4] [n. d.]. Fast and reliable end-to-end testing for modern web apps | Playwright — playwright.dev. <https://playwright.dev/>. [Accessed 06-02-2026].
- [5] [n. d.]. Industry-Focused Job Boards | iHire — ihire.com. <https://www.ihire.com/communities>. [Accessed 10-10-2025].
- [6] [n. d.]. LM Studio - Local AI on your computer — lmstudio.ai. <https://lmstudio.ai/>. [Accessed 06-02-2026].
- [7] 2016. General Data Protection Regulation (GDPR). <https://gdpr-info.eu/>. Regulation (EU) 2016/679.

- [8] 2023. A Simplified Regulatory Checklist for Financial Institutions. <https://arcticwolf.com/blog/a-simplified-regulatory-checklist-for-financial-institutions/>. States that financial institutions face a regulatory and security compliance burden exceeding that of most other industries.
- [9] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. 2025. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925* (2025).
- [10] Nasser Mohammed Al-Fannah, Wanpeng Li, and Chris J Mitchell. 2018. Beyond cookie monster amnesia: Real world persistent online tracking. In *International Conference on Information Security*. Springer, 481–501.
- [11] Adam Bates, Dave (Jing) Tian, Kevin R.B. Butler, and Thomas Moyer. 2015. Trustworthy Whole-System Provenance for the Linux Kernel. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, D.C., 319–334. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates>
- [12] Md Hajian Berenjestanaki, Mauro Conti, and Ankit Gangwal. 2019. On the exploitation of online SMS receiving services to forge ID verification. In *Proceedings of the 14th International Conference on Availability, Reliability and Security*. 1–5.
- [13] Evangelos Bitsikas, Theodor Schnitzler, Christina Pöpper, and Aanjan Ranganathan. 2023. Freaky leaky {SMS}: Extracting user locations by analyzing {SMS} timings. In *32nd USENIX Security Symposium (USENIX Security 23)*.
- [14] Shuai Cheng, Shu Meng, Haitao Xu, Haoran Zhang, Shuai Hao, Chuan Yue, Wenrui Ma, Meng Han, Fan Zhang, and Zhao Li. 2025. Effective PII Extraction from LLMs through Augmented Few-Shot Learning. In *34th USENIX Security Symposium (USENIX Security 25)*. 8155–8173.
- [15] Yanan Cheng, Han Wang, Zhaoxin Zhang, and Ning Li. 2020. Characterizing the security threats of disposable phone numbers. In *International Conference on Frontiers in Cyber Security*. Springer, 491–507.
- [16] Cheng Cui, Ting Sun, Manhui Lin, Tingquan Gao, Yubo Zhang, Jiaxuan Liu, Xueqing Wang, Zelin Zhang, Changda Zhou, Hongen Liu, et al. 2025. Paddleocr 3.0 technical report. *arXiv preprint arXiv:2507.05595* (2025).
- [17] Cybersecurity and Infrastructure Security Agency (CISA). [n.d.]. Critical Infrastructure Sectors | CISA — cisa.gov. <https://www.cisa.gov/topics/critical-infrastructure-security-and-resilience/critical-infrastructure-sectors>. [Accessed 14-11-2025].
- [18] Avisha Das, Shahryar Baki, Ayman El Aassal, Rakesh Verma, and Arthur Dunbar. 2019. SoK: a comprehensive reexamination of phishing research from the security perspective. *IEEE Communications Surveys & Tutorials* 22, 1 (2019), 671–708.
- [19] Cynthia Dwork, Adam Smith, Thomas Steinke, and Jonathan Ullman. 2017. Exposed! a survey of attacks on private data. *Annual Review of Statistics and Its Application* 4, 1 (2017).
- [20] Forcepoint. 2025. ThreatSeeker URL Categories and Description. <https://help.forcepoint.com/fpone/migration/webtothreatseekerurl/guid-b8ac3928-a64c-4468-bf16-4a6d4932c2c7.html>.
- [21] Yuhao Gao, Guoai Xu, Li Li, Xiapu Luo, Chenyu Wang, and Yulei Sui. 2022. Demystifying the underground ecosystem of account registration bots. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 897–909.
- [22] Anthony Gavazzi, Ryan Williams, Engin Kirda, Long Lu, Andre King, Andy Davis, and Tim Leek. 2023. A study of {Multi-Factor} and {Risk-Based} authentication availability. In *32nd USENIX Security Symposium (USENIX Security 23)*. 2043–2060.
- [23] Martin Georgiev and Vitaly Shmatikov. 2016. Gone in six characters: Short urls considered harmful for cloud services. *arXiv preprint arXiv:1604.02734* (2016).
- [24] Florian Hantke, Sebastian Roth, Rafael Mrowczynski, Christine Utz, and Ben Stock. 2024. Where are the red lines? towards ethical server-side scans in security and privacy research. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4405–4423.
- [25] Fannv He, Yan Jia, Jiayu Zhao, Yue Fang, Jice Wang, Mengyue Feng, Peng Liu, and Yuqing Zhang. 2024. Maginot line: Assessing a new cross-app threat to pii-as-factor authentication in chinese mobile apps. In *Network and Distributed System Security Symposium (NDSS)*.
- [26] Byeongdo Hong, Sangwook Bae, and Yongdae Kim. 2018. GUTI Reallocation Demystified: Cellular Location Tracking with Changing Temporary Identifier.. In *NDSS*.
- [27] Syed Rafiul Hussain, Mitziu Echeverria, Omar Chowdhury, Ninghui Li, and Elisa Bertino. 2019. Privacy attacks to the 4G and 5G cellular paging protocols using side channel information. *Network and distributed systems security (NDSS) symposium2019* (2019).
- [28] Ziwei Ji, Tiezheng Yu, Yan Xu, Nayeon Lee, Etsuko Ishii, and Pascale Fung. 2023. Towards mitigating LLM hallucination via self reflection. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 1827–1843.
- [29] Soheil Khodayari, Kai Glauber, and Giancarlo Pellegrino. 2025. Do (Not) Follow the White Rabbit: Challenging the Myth of Harmless Open Redirection. In *Proceedings 2025 Network and Distributed System Security Symposium*. Internet Society, San Diego, CA, USA. doi:10.14722/ndss.2025.240523
- [30] Myounghoon Kim, Joon Suh, and Hunyeong Kwon. 2022. A study of the emerging trends in SIM swapping crime and effective countermeasures. In *2022 IEEE/ACIS 7th International Conference on Big Data, Cloud Computing, and Data Science (BCD)*. IEEE, 240–245.
- [31] Balachander Krishnamurthy and Craig E Wills. 2009. On the leakage of personally identifiable information via online social networks. In *Proceedings of the 2nd ACM workshop on Online social networks*. 7–12.
- [32] Nitya Lakshmanan, Nishant Budhdev, Min Suk Kang, Mun Choon Chan, and Jun Han. 2021. A stealthy location identification attack exploiting carrier aggregation in cellular networks. In *30th usenix security symposium (usenix security 21)*.
- [33] Leona Lassak, Elleen Pan, Blase Ur, and Maximilian Golla. 2024. Why aren't we using passkeys? Obstacles companies face deploying {FIDO2} passwordless authentication. In *33rd USENIX Security Symposium (USENIX Security 24)*. 7231–7248.
- [34] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. 2019. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. In *Network and Distributed System Security (NDSS) Symposium 2019*. Internet Society. doi:10.14722/ndss.2019.23386
- [35] Yuyei Liu, Yuqi Jia, Jinyuan Jia, and Neil Zhenqiang Gong. 2025. Evaluating {LLM-based} Personal Information Extraction and Countermeasures. In *34th USENIX Security Symposium (USENIX Security 25)*. 1669–1688.
- [36] Lutfun Nahar Lota and BM Mainul Hossain. 2017. A systematic literature review on sms spam detection techniques. *International Journal of Information Technology and Computer Science (IJITCS)* 9, 7 (2017), 42–50.
- [37] Juniper Lovato, Philip Mueller, Parisa Suchdev, and Peter Dodds. 2023. More data types more problems: A temporal analysis of complexity, stability, and sensitivity in privacy policies. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency*. 1088–1100.
- [38] Srdjan Matic, Costas Iordanou, Georgios Smaragdakis, and Nikolaos Laoutaris. 2020. Identifying Sensitive URLs at Web-Scale. In *Proceedings of the ACM Internet Measurement Conference*. ACM, Virtual Event USA, 619–633. doi:10.1145/3419394.3423653
- [39] Brad Miller, Ling Huang, A. D. Joseph, and J. D. Tygar. 2014. I Know Why You Went to the Clinic: Risks and Realization of HTTPS Traffic Analysis. In *Privacy Enhancing Technologies*.
- [40] MITRE Corporation. 2024. CWE-331: Insufficient Entropy. <https://cwe.mitre.org/data/definitions/331.html>. Accessed: 2025-09-18.
- [41] José Miguel Moreno, Srdjan Matic, Narseo Vallina-Rodriguez, and Juan Tapiador. 2023. Your Code is 0000: An Analysis of the Disposable Phone Numbers Ecosystem. In *2023 7th Network Traffic Measurement and Analysis Conference (TMA)*. IEEE, 1–10.
- [42] Ilona Murymets and Roger Piqueras Jover. 2012. Crime scene investigation: SMS spam data analysis. In *Proceedings of the 2012 Internet Measurement Conference*. 441–452.
- [43] Aleksandr Nahapetyan, Sathvik Prasad, Kevin Childs, Adam Oest, Yeganeh Ladwig, Alexandros Kapravelos, and Bradley Reaves. 2024. On sms phishing tactics and infrastructure. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE.
- [44] National Institute of Standards and Technology (NIST). 2010. Guide to Protecting the Confidentiality of Personally Identifiable Information (PII). <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-122.pdf>. Special Publication 800-122, Revision 1.
- [45] Alexander Neumann, Johannes Barnickel, and Ulrike Meyer. 2010. Security and privacy implications of url shortening services. In *Proceedings of the Workshop on Web 2.0 Security and Privacy*.
- [46] Nick Nikiforakis, Marco Balduzzi, Steven Van Acker, Wouter Joosen, and Davide Balzarotti. 2011. Exposing the lack of privacy in file hosting services. In *4th USENIX Workshop on Large-Scale Exploits and Emergent Threats (LEET 11)*.
- [47] David W Opderbeck. 2022. Cybersecurity and Data Breach Harms: Theory and Reality. *Md. L. Rev.* 82 (2022).
- [48] OWASP Foundation. [n.d.]. Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html#idle-timeout. OWASP Cheat Sheet Series, accessed 2025-11-13.
- [49] OWASP Foundation. 2025. Insufficient Session ID Length. https://owasp.org/www-community/vulnerabilities/Insufficient_Session-ID_Length. Author: Jake Karnes; Contributor(s): Greg Heartsfield; Accessed: 2025-09-18.
- [50] Lianglu Pan, Shaanan Cohnen, Toby Murray, and Van-Thuan Pham. 2024. EDE-Fuzz: A web API fuzzer for excessive data exposures. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–12.
- [51] Sazzadur Rahman, Gang Wang, and Danfeng Yao. 2019. Security Certification in Payment Card Industry: Testbeds, Measurements, and Recommendations. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 481–498.
- [52] Md Lutfur Rahman, Daniel Timko, Hamid Wali, and Ajaya Neupane. 2023. Users really do respond to smishing. In *Proceedings of the thirteenth ACM conference on data and application security and privacy*. 49–60.
- [53] Harshit Rajgarhia, Suryam Gupta, Asif Shaik, Gulipalli Praveen Kumar, Y Santhoshraj, Sanka Nithya Tanvy Nishitha, and Abhishek Mukherji. [n.d.]. An Evaluation Study of Hybrid Methods for Multilingual PII Detection. In *NeurIPS 2025 Workshop on Evaluating the Evolving LLM Lifecycle: Benchmarks, Emergent Abilities, and Scaling*.

- [54] Jannis Rautenstrauch and Ben Stock. 2024. Who's Breaking the Rules? Studying Conformance to the HTTP Specifications and its Security Impact. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. 843–855.
- [55] Bradley Reaves, Logan Blue, Dave Tian, Patrick Traynor, and Kevin RB Butler. 2016. Detecting SMS spam in the age of legitimate bulk messaging. In *Proceedings of the 9th ACM Conference on Security & Privacy in Wireless and Mobile Networks*. 165–170.
- [56] Bradley Reaves, Nolen Scaife, Dave Tian, Logan Blue, Patrick Traynor, and Kevin RB Butler. 2016. Sending out an SMS: Characterizing the Security of the SMS Ecosystem with Public Gateways. In *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 339–356.
- [57] Bradley Reaves, Luis Vargas, Nolen Scaife, Dave Tian, Logan Blue, Patrick Traynor, and Kevin RB Butler. 2018. Characterizing the security of the SMS ecosystem with public gateways. *ACM Transactions on Privacy and Security (TOPS)* 22, 1 (2018), 1–31.
- [58] Jingjing Ren, Ashwin Rao, Martina Lindorfer, Arnaud Legout, and David Choffnes. 2016. Recon: Revealing and controlling pii leaks in mobile network traffic. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services*. 361–374.
- [59] Doyen Sahoo, Chenghao Liu, and Steven CH Hoi. 2017. Malicious URL detection using machine learning: A survey. *arXiv preprint arXiv:1701.07179* (2017).
- [60] M Angela Sasse, Michelle Steves, Kat Krol, and Dana Chisnell. 2014. The great authentication fatigue—and how to overcome it. In *International Conference on Cross-Cultural Design*. Springer, 228–239.
- [61] Asuman Senol, Gunes Acar, Mathias Humbert, and Frederik Zuiderveen Borgesius. 2022. Leaky forms: A study of email and password exfiltration before form submission. In *31st USENIX Security Symposium (USENIX Security 22)*. 1813–1830.
- [62] I Shafi, MS Abd Latiff, Haruna Chiroma, Oluwafemi Osho, G Abdul-Salaam, AI Abubakar, and T Herawan. 2017. A review on mobile SMS spam filtering techniques. *IEEE Access* 5 (2017), 15650–15666.
- [63] Wenni Syafitri, Zarina Shukur, Umi Asma Mokhtar, Rossilawati Sulaiman, and Muhammad Azwan Ibrahim. 2022. Social engineering attacks prevention: A systematic literature review. *IEEE access* 10 (2022), 39325–39343.
- [64] OWASP API Security Project team. 2019. API3:2019 Excessive Data Exposure - OWASP API Security Top 10 — owasp.org. [https://owasp.org/{A}\[P\]{I}-{S}ecurity/editions/2019/en/0xa3-excessive-data-exposure/](https://owasp.org/{A}[P]{I}-{S}ecurity/editions/2019/en/0xa3-excessive-data-exposure/). [Accessed 29-04-2026].
- [65] OWASP API Security Project team. 2023. API5:2023 Broken Function Level Authorization - OWASP API Security Top 10 — owasp.org. [https://owasp.org/{A}\[P\]{I}-{S}ecurity/editions/2023/en/0xa5-broken-function-level-authorization/](https://owasp.org/{A}[P]{I}-{S}ecurity/editions/2023/en/0xa5-broken-function-level-authorization/). [Accessed 06-02-2026].
- [66] Daniel Timko and Muhammad Lutfur Rahman. 2023. Commercial anti-smishing tools and their comparative effectiveness against modern threats. In *Proceedings of the 16th ACM Conference on Security and Privacy in Wireless and Mobile Networks*. 1–12.
- [67] Kaleem Ullah, Imran Rashid, Hammad Afzal, Mian Muhammad Waseem Iqbal, Yawar Abbas Bangash, and Haider Abbas. 2020. SS7 vulnerabilities—a survey and implementation of machine learning vs rule based filtering for detection of SS7 network attacks. *IEEE Communications Surveys & Tutorials* 22, 2 (2020), 1337–1371.
- [68] Rick van der Kleij, Susanne van 't Hoff—De Goede, Steve van de Weijer, and Rutger Leukfeldt. 2023. Social engineering and the disclosure of personal identifiable information: Examining the relationship and moderating factors using a population-based survey experiment. *Journal of Criminology* 56, 2-3 (2023), 278–293.
- [69] Wayne Wang, Aaron Ortwein, Enrique Sobrados, Robert Stanley, Piyush Kumar Sharma, Afsah Anwar, and Roya Ensafi. 2026. MVPNalyzer: An Investigative Framework for Auditing the Security & Privacy of Mobile VPNs. In *The Network and Distributed System Security Symposium*.
- [70] Xueqiang Wang, Yuqiong Sun, Susanta Nanda, and Xiaofeng Wang. 2023. Credit karma: Understanding security implications of exposed cloud services through automated capability inference. In *32nd USENIX Security Symposium (USENIX Security 23)*. 6007–6024.
- [71] Andrew G West and Adam J Aviv. 2014. On the Privacy Concerns of URL Query Strings. In *Web 2.0 Security & Privacy*.
- [72] Zack Whittaker. 2019. Millions of SMS messages exposed in database security lapse. <https://techcrunch.com/2019/12/01/millions-sms-messages-exposed/>. Accessed: 2025-11-12.

A Open Science

The artifacts of this paper are publicly available at https://github.com/cs-maestro/cost_of_convenience. The repository contains scripts and non-sensitive artifacts supporting URL-artifact collection, pre-processing, filtering, PII detection, validation, and analysis. We do not provide SMS-gateway scraping scripts that could be misused to

gather messages containing user PII or URLs that lead to user PII. In addition, we do not release raw datasets, SMS messages, URLs, or identifiers that contain, or could be reverse-engineered to expose, private data. We also withhold these materials because we have signed NDAs with some service providers or they have specifically requested anonymity, and because the majority of affected services have not yet remediated the underlying issues.

B Ethical Considerations

This study was reviewed and approved by our institution's ethics board (IRB). The primary stakeholders are users whose data appeared in SMS-delivered URLs, other users of vulnerable services, service providers, public SMS gateway operators, and society at large. Although prior work has treated messages on public SMS gateways as public data [56, 57], we treated exposed data as sensitive and minimized interaction with live systems. We collected messages only from **public** SMS gateways that exposed them without authentication or technical access controls and used conservative crawlers consistent with prior measurement practices [24, 54]. We collected only the minimum data needed to determine whether PII was exposed, and all raw messages, URLs, and screenshots were ultimately deleted at the end of the study. We do not release raw SMS messages, vulnerable URLs, non-redacted screenshots, extracted PII, or SMS-gateway scraping scripts. PII detection, OCR, and LLM analysis were performed locally, so sensitive artifacts were not transmitted to third-party analysis services. Enumeration and authentication testing was also strictly bounded. Confirmed vulnerabilities were disclosed through providers' published contact or security channels, with at least 45 days provided for remediation.

C Public SMS Gateways

azroTV.com, bestsms.xyz, clearcode.cn, freeonlinephone.org, freephonenum.com, getsms.cc, lothelper.com, zh.mytrashmobile.com, receive-a-sms.com, receivefreemails.net, receive-sms.cc, receivesms.co, receive-sms-free.net, receivesmsonline.net, receivesmsonline.in, receive-smss.com, shejiinn.com, smstome.com, storytrain.info, temp-number.com, temporary-phone-number.com, us-phone-number.com, xinghai.party, zusms.com

D PII Taxonomy

mothers maiden name, password, security question, user name, products purchased, psychological trends, search history, services purchased, email address, email content, postal address, telephone number, text messages, citizenship, color, date of birth, education, employment history, family, first name, gender, gender identity, hair color, height, language, marital status, military or veteran status, place of birth, race, religion, sex life, union membership, immigration status, last name, bank account number, credit card number, debit card number, financial account number, insurance policy number, driver authorization card number, health insurance identification number, individual taxpayer identification number, medical identification number, military identification card number, nondriver state identification card number, passport number, social security number, state identification card number, drivers license number, aids, breastfeeding, cancer, childbirth, chiropractic, diagnosis, disability, health records, medicine, mental condition,

pregnancy, genetic data, hiv, request for family care leave, request for leave for an employees own serious health condition, location, records of personal property

E Case Study Figures

All vulnerabilities illustrated in this appendix were responsibly disclosed and patched before publication. We include only remediated cases and redact all user-identifying information.

Figure 3: Anonymized case study

Figure 4: everquote.com

Figure 5: ihire.com

```

"text": "\u0022consumer_id\u0022:\u0022\u0022,\u0022first_name\u0022:\u0022\u0022,\u0022last_name\u0022:\u0022\u0022,\u0022routing_number\u0022:\u0022\u0022,\u0022account_number\u0022:\u0022\u0022,\u0022ssn\u0022:\u0022\u0022,\u0022email\u0022:\u0022\u0022,\u0022street_address\u0022:\u0022\u0022,\u0022street_address2\u0022:\u0022\u0022,\u0022city\u0022:\u0022Louisville\u0022,\u0022state_code\u0022:\u0022\u0022,\u0022zip_code\u0022:\u0022\u0022,\u0022home_phone\u0022:\u0022\u0022,\u0022mobile_phone\u0022:\u0022\u0022,\u0022reference_date\u0022:\u00222025-10-18\u0022\""

```

Figure 6: Anonymized case study

Figure 7: dimorder.com

Figure 8: stickermule.com